SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: Total Marks: 30

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
 <input type="text" name="name">
 <input type="email" name="email">
 <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
 <head>
 <title>Company</title>
 </head>
 <body>
 <div>
 <h1>Welcome
 <p>About us
 </div>
 </body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
 <input type="text" name="username">
 <input type="password" name="password">
 <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Code of Conduct:

I **B. Maheshwar (192411195)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: B. Maheshwar

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

CSA4305- INTERNET PROGRAMING
Assessment tool - 1

1. Online Symposium Portal

1. Identify appropriate HTML5 semantic elements missing in the structure.

The given HTML form works correctly, but it does not use HTML5 semantic elements that improve the structure and readability of the webpage. The missing semantic elements include <header>, <main>, <section>, and <footer>. The form should also include a heading using <h1> or <h2> to indicate the purpose of the page. In addition, every input field should have a corresponding <label> element to improve accessibility. Using semantic elements helps browsers, search engines, and assistive technologies understand the content more effectively while making the webpage easier to maintain.

2. Modify the structure to make it standards-compliant.

A standards-compliant HTML structure is shown below:

```
<!DOCTYPE html><html><head><title>Symposium Registration</title></head>
<body>
<header><h1>Online Symposium Registration</h1></header>
<main>
<section>
<form action="/register" method="POST">
<label for="name">Name:</label><input type="text" id="name" name="name" required>
<label for="email">Email:</label><input type="email" id="email" name="email" required>
<button type="submit">Submit</button>
</form>
</section>
</main>
<footer>
<p>© 2026 College Symposium</p>
</footer>
</body></html>
```

This version follows HTML5 standards by using semantic elements, labels for form controls, the required attribute for validation, and an explicit submit button.

3. Interpret the HTTP request generated during form submission.

When the user clicks the **Submit** button, the browser collects the values entered in the form fields and creates an HTTP POST request. Since the method is **POST**, the form data is sent inside the request body instead of the URL. The browser also includes HTTP headers such as Host, Content-Type, Content-Length, User-Agent, and Accept. The request is then transmitted to the server at the specified action URL (/register). The server receives the data, processes the registration details, and stores the information in the database if validation is successful.

4. Analyze the HTTP response cycle from server to browser.

After receiving the request, the server validates the submitted information. If the data is valid, the server stores it and generates an HTTP response, usually with a **200 OK** or **201 Created** status code. If validation fails, it may return a **400 Bad Request** response along with appropriate error messages. The browser receives the response, interprets the status code, downloads any required resources, and updates the webpage by displaying either a success confirmation or validation errors. This request-response cycle forms the foundation of communication between web browsers and servers.

5. Suggest improvements for usability and accessibility.

Several improvements can make the form more user-friendly and accessible:

- Add labels for every input field.
- Use the **required** attribute for mandatory fields.
- Provide placeholder text and validation messages.
- Use semantic HTML elements such as <header>, <main>, and <section>.
- Add ARIA attributes for users with disabilities.
- Display confirmation messages after successful registration.
- Ensure the form is responsive for mobile devices.
- Use HTTPS to securely transmit user information.

2. Cross-Browser Compatibility Issue

1. Identify improper nesting and missing closing tags.

The HTML contains several structural errors. The <h1> tag is missing its closing </h1> tag, and the tag is missing the closing tag. The element is closed before these elements are explicitly closed, resulting in improper nesting. Such errors make the HTML invalid according to W3C standards and may cause browsers to interpret the document differently.

2. Analyze how different browsers may interpret this structure.

Modern browsers such as Chrome, Edge, Firefox, and Safari automatically correct many HTML errors while constructing the Document Object Model (DOM). They may insert missing closing tags automatically so the webpage appears normal. However, the correction process is browser-dependent, meaning different browsers may produce slightly different layouts. This inconsistency can affect CSS styling, JavaScript execution, and webpage accessibility.

3. Identify missing semantic elements (e.g., header, section).

The webpage should include HTML5 semantic elements such as:

- <header> for the page heading.
- <main> for the main content.
- <section> to group related information.
- <footer> for footer information.

Using these semantic elements improves code readability, search engine optimization (SEO), accessibility, and overall webpage organization.

4. Provide a corrected W3C-compliant HTML structure.

<!DOCTYPE html><html>
<head><title>Company</title></head>
<body>

```
<header><h1>Welcome</h1></header>
<main>
<section><p>About us</p></section>
</main>
<footer><p>© Company</p></footer>
</body>
</html>
```

This version correctly closes every HTML element and uses semantic tags recommended by HTML5.

5. Suggest techniques to ensure cross-browser compatibility.

To improve browser compatibility:

- Follow W3C HTML and CSS standards.
- Validate HTML using online validators.
- Test the webpage in multiple browsers.
- Use CSS resets or Normalize.css.
- Avoid deprecated HTML elements.
- Use responsive web design techniques.
- Use feature detection instead of browser detection.
- Keep browsers and development tools updated.

3. Form Submission Failure

1. Identify issues in the form configuration.

The form contains two major issues. First, it uses the **GET** method to transmit login credentials, exposing usernames and passwords in the URL. Second, the button type is **button** instead of **submit**, so clicking the button does not automatically submit the form. These configuration errors prevent proper login functionality and reduce security.

2. Analyze why the HTTP request is not triggered.

The HTTP request is not sent because the button is defined as:

```
<button type="button">Login</button>
```

A button with type="button" performs no action unless JavaScript is explicitly written to submit the form. Only a button with type="submit" automatically sends the form data to the server when clicked.

3. Interpret the role of GET method in this scenario.

The GET method appends form data to the URL as query parameters. This makes the username and password visible in the browser's address bar, browser history, bookmarks, proxy logs, and server logs. GET is intended for retrieving information rather than sending confidential user credentials. Therefore, it is not appropriate for login forms.

4. Propose a corrected implementation.

```
<!DOCTYPE html><html>
<head><title>Login</title></head>
<body>
<form action="/submit" method="POST">
<label for="username">Username:</label><input type="text" id="username" name="username" required>
<label for="password">Password:</label><input type="password" id="password" name="password" required>
<button type="submit">Login</button>
</form>
</body>
</html>
```

This implementation uses the **POST** method, includes proper labels, uses the **required** attribute, and replaces the button type with **submit**, ensuring successful form submission.

5. Suggest improvements for secure data transmission.

To improve login security:

- Use the **POST** method instead of GET.
- Enable **HTTPS** to encrypt communication.
- Store passwords using strong hashing algorithms such as **bcrypt** or **Argon2**.
- Validate all user input on both the client and server.
- Protect against SQL Injection using prepared statements.
- Protect against Cross-Site Scripting (XSS).
- Implement secure session management.
- Enable Multi-Factor Authentication (MFA).

- Use strong password policies and account lockout mechanisms after repeated failed login attempts.

These practices significantly improve the security and reliability of web applications.